A SERVICE-ORIENTED TASK & DEADLINE MANAGEMENT PLATFORM

A Mini Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

Madhupraveena - 2420030321

Hema Thulasi - 2420030699

Ranan Magdala - 2420030744

Under the Esteemed Guidance of

Dr.Neelima Gogineni

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

KONERU LAKSHMAIAH EDUCATION FOUNDATION
(Deemed to be University), Bowrampet, Hyderabad – 500043,
Telangana, India
Academic Year 2026–2027

K L (DEEMED TO BE) UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We hereby declare that the Mini Project Report entitled “A Service-Oriented Task & Deadline Management Platform” is a bona fide record of the work carried out by the members of Team 11, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at K L (Deemed to be) University, during the academic year 2026–2027. The work presented in this report is prepared by our team. All external resources, tools, and references used in the preparation and development of the project are acknowledged appropriately.

We further declare that the implementation details and results included in this report will be verified against the project source code and actual testing evidence before final submission.

MADHU PRAVEENA – 2420030321

Ranan Magdala - 2420030744

Hema Thulasi - 2420030699

Place: Hyderabad

Date: _____

K L (DEEMED TO BE) UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the Mini Project Report entitled “A Service-Oriented Task & Deadline Management Platform” is a bona fide work done and submitted by MADHU PRAVINA (24200303) and the other members of Team 11, in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science and Engineering, K L (Deemed to be) University, during the academic year 2026–2027.

The work presented in this report has been carried out under the guidance and supervision of the undersigned. The report is recommended for evaluation as part of the Software-Oriented Architecture (SOA) project course.

Signature of the Course Co-ordinator	**Signature of HOD**
Name: _____ Date: _____	Name: _____ Date: _____

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the guidance, encouragement, and support of several individuals. We express our sincere gratitude to everyone who contributed directly or indirectly to the completion of our project, “A Service-Oriented Task & Deadline Management Platform.”

We extend our deepest appreciation to our project guide, _____, Department of Computer Science and Engineering, for valuable guidance, constructive feedback, and continuous encouragement throughout the planning, design, and development of this project.

We express our gratitude to the Head of the Department and the faculty members of the Department of Computer Science and Engineering, K L (Deemed to be) University, for providing the academic environment, infrastructure, and resources required to carry out this work.

We also thank the teaching and non-teaching staff members for their assistance. Finally, we thank our parents, friends, and classmates for their support, patience, and encouragement during the project.

MADHU PRAVEENA – 2420030321

Ranan Magdala - 2420030744

Hema Thulasi - 2420030699

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Abstract	6
2	Introduction	7
3	Literature Survey	8
4	Problem Statement and Objectives	9
5	Hardware and Software Requirements	10
6	Implementation	11
7	Results and Discussion	12
8	Conclusion and Future Scope	13
9	References	14
Appendix A	Detailed System Architecture	15
Appendix B	Data Design and Validation	16
Appendix C	REST API Design Reference	17
Appendix D	Scheduling and Notification Workflow	18
Appendix E	Test Plan and Evaluation	19
Appendix F	Installation, Configuration and Deployment	20–21

Note: Page numbers correspond to the section sequence in this report. Update them if additional pages or screenshots are inserted.

1. ABSTRACT

Task and deadline management is an essential activity for students and professionals who handle multiple assignments, meetings, milestones, and personal responsibilities. When task information is distributed across notebooks, messages, and separate applications, users may lose track of priorities or overlook approaching deadlines. This project presents a Service-Oriented Task & Deadline Management Platform that aims to centralize task organization and support timely deadline notifications through a web-based interface. The proposed application uses React.js and Tailwind CSS for the frontend, Node.js and Express.js for backend services, and MongoDB for data persistence. JWT is included for token-based authentication. Node-Cron is used to schedule deadline checks, while Nodemailer and Socket.IO provide email and real-time notification capabilities. The system exposes REST APIs to connect the frontend with backend operations. Docker, GitHub, and Postman support packaging, source-code collaboration, and API validation. The report describes the project motivation, requirements, architecture, implementation modules, testing plan, and future enhancements. Actual screenshots and measured test results should be inserted from the completed application before final evaluation.

Team Name

Team 11 – SOA Project

Project Title

A Service-Oriented Task & Deadline Management Platform

Team Logo

No separate team logo supplied. The institutional KL University logo is displayed on the cover page.

Team Members

S. No.	Name	Roll Number	Role
1	Madhu Praveena	2420030321	Member
2	Hema	2420030699	Member
3	Ranan	2420030744	Member

2. INTRODUCTION

The increasing use of digital tools in academic and professional environments has created a need for reliable ways to organize activities and monitor deadlines. Students may need to coordinate assignments, laboratory submissions, examinations, project milestones, and team activities at the same time. Professionals face similar challenges when handling meetings, deliverables, and time-bound responsibilities. When information is maintained in separate applications or informal notes, it becomes difficult to obtain a consolidated view of pending work.

A task-management platform provides a centralized environment in which users can record tasks, maintain descriptions, assign due dates, and track progress. However, task storage alone does not ensure that users act on time. A useful platform should also support deadline monitoring and notify users when important dates approach. The proposed project combines task-management operations with scheduled processing and multiple notification channels.

The platform is designed as a web application. React.js provides the interactive user interface, while Tailwind CSS supports responsive styling. The frontend communicates with a Node.js and Express.js backend through REST APIs. MongoDB is used for persistent storage. JWT supports authenticated access to protected resources. A scheduled component built using Node-Cron evaluates deadline conditions. Nodemailer is intended for email delivery, and Socket.IO supports real-time events to connected clients.

The project adopts a service-oriented organization of responsibilities. User-interface logic, API handling, authentication, data persistence, scheduling, and notification delivery are treated as distinct concerns connected through defined interfaces. This organization makes the application easier to understand, test, and extend. The term service-oriented in this report refers to modular service responsibilities; it does not imply that each component is necessarily deployed as a separate microservice.

2.1 Objectives

- Centralize task and deadline information in a web-based application.
- Provide authenticated access to user-specific task operations.
- Support task creation, retrieval, modification, and deletion according to implemented functionality.
- Evaluate due dates through scheduled background processing.
- Provide email and real-time notification mechanisms.
- Use modular REST-based interfaces to separate frontend and backend responsibilities.
- Support development, testing, and deployment using GitHub, Postman, and Docker where configured.

2.2 Organization of the Report

The remaining sections review related task-management systems, define the problem and objectives, list hardware and software requirements, describe the implementation modules, discuss the evaluation approach, and conclude with future enhancements.

3. LITERATURE SURVEY

Todoist

Todoist provides task capture, due dates, recurring tasks, priorities, and project organization. Its feature model demonstrates the value of combining task details with a structured view of upcoming work. The proposed project focuses on a smaller academic implementation with deadline checks and notification integration.

Reference: https://todoist.com/

Microsoft To Do

Microsoft To Do supports personal task lists, reminders, due dates, and task grouping. It illustrates how a simple interface can help users maintain daily priorities. The proposed platform similarly aims to provide a centralized task view, while emphasizing its own Node.js-based service architecture.

Reference: https://to-do.office.com/

Trello

Trello organizes work using boards, lists, and cards. It demonstrates a visual approach to tracking work stages and collaboration. The current project has a narrower scope focused on task and deadline management rather than assuming full Kanban collaboration features.

Reference: https://trello.com/

Asana

Asana supports project planning, task assignment, due dates, and progress tracking. Its workflow-oriented design highlights the importance of clear ownership and task status. The proposed platform adopts task lifecycle concepts but does not claim equivalent enterprise project-management capabilities.

Reference: https://asana.com/

ClickUp

ClickUp combines tasks, views, reminders, and productivity features in a unified workspace. It provides a reference for presenting multiple task attributes together. The proposed project focuses on a compact implementation using React, Express, and MongoDB.

Reference: https://clickup.com/

Any.do

Any.do offers task lists, reminders, and calendar-oriented planning. It demonstrates the role of reminders in supporting timely task completion. The proposed platform includes scheduled deadline evaluation and email notification as part of its intended workflow.

Reference: https://www.any.do/

Google Tasks

Google Tasks provides task creation, due dates, and integration with Google's productivity ecosystem. Its lightweight task model is relevant to the project's focus on clear task records and due-date tracking. The proposed platform uses its own database and REST API.

Reference: https://tasks.google.com/

Notion

Notion supports databases and customizable task or project views. It demonstrates how structured records can be used to organize different types of work. The proposed application uses a more focused task schema and dedicated deadline-notification components.

Reference: https://www.notion.so/

Jira

Jira provides issue tracking, workflows, and project-level task management, particularly for software teams. It highlights the importance of status transitions and traceable work items. The proposed platform is intended as a smaller task-management solution for individual use and academic project work.

Reference: https://www.atlassian.com/software/jira

TickTick

TickTick combines task lists, reminders, calendar views, and productivity tools. It reinforces the importance of accessible due-date information and reminder channels. The proposed project explores similar core needs through a custom full-stack implementation.

Reference: https://ticktick.com/

4. PROBLEM STATEMENT AND OBJECTIVES

4.1 Problem Statement

Users need a centralized and reliable way to organize tasks and receive timely alerts for approaching deadlines, because scattered task records and manual reminders can lead to missed or delayed work.

4.2 Problem Description

Academic and personal tasks are often tracked through disconnected tools, making it difficult to review priorities and upcoming due dates in one place. Without automated deadline monitoring and notification support, users must repeatedly check task lists and may overlook important activities.

4.3 Proposed Solution

The proposed platform provides a web interface for managing tasks and a backend that coordinates authentication, persistence, scheduled deadline checks, and notification delivery. REST APIs connect the frontend to backend services. Scheduled checks are intended to identify tasks that meet configured reminder conditions and invoke email or real-time notification mechanisms.

4.4 Project Objectives

- Design a responsive task-management interface using React.js and Tailwind CSS.
- Develop backend endpoints using Node.js and Express.js.
- Store user and task information in MongoDB.
- Use JWT to authenticate protected operations.
- Schedule deadline evaluation using Node-Cron.
- Integrate Nodemailer for email notifications.
- Integrate Socket.IO for real-time notification events.
- Test API behavior and application workflows using Postman and documented test cases.
- Use Docker for repeatable deployment if container configuration is included in the repository.

4.5 Expected Outcomes

The expected outcome is a modular web application that enables users to manage task information and receive deadline-related notifications through configured channels. The outcome must be demonstrated using the actual running application, test evidence, and screenshots. Performance claims should be added only after measurements are collected.

5. HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

Component	Minimum / recommended requirement
Processor	Modern dual-core processor or better
RAM	4 GB minimum; 8 GB recommended for development
Storage	At least 2 GB free space for source code and dependencies; additional space for tools
Display	1366 × 768 resolution or higher recommended
Network	Internet or network access for package installation, database connectivity, and email service

5.2 Software Requirements

Software / technology	Purpose
Operating System	Windows, Linux, or macOS development environment
Node.js and npm	Backend runtime and package management
React.js	Frontend user-interface development
Tailwind CSS	Responsive interface styling
Express.js	Backend routing and middleware
MongoDB	Persistent data storage
JWT library	Token generation and verification
Node-Cron	Scheduled task execution
Nodemailer	Email transport and delivery
Socket.IO	Real-time client-server events
Docker	Container packaging, if configured
Git and GitHub	Version control and team collaboration
Postman	REST API testing
Modern web browser	Application access and UI testing

5.3 External Services and Configuration

The application may require a MongoDB connection string and an SMTP-compatible email account. These values should be configured through environment variables. JWT signing secrets and mail credentials must not be committed to the public repository. The exact variable names should be taken from the backend configuration files.

6. IMPLEMENTATION

6.1 Frontend Implementation

The frontend is developed using React.js and Tailwind CSS. React components are responsible for rendering task information, collecting user input, and presenting operation results. The interface communicates with backend endpoints through HTTP requests and processes JSON responses. The exact pages and component names should be documented from the source repository.

- Authentication screens, if implemented, allow users to register or sign in.
- Task views present stored task records and relevant task attributes.
- Task forms collect the fields required by the backend schema.
- Notification UI displays real-time events when the client is connected and subscribed.

6.2 Backend Implementation

Node.js provides the runtime for the backend, and Express.js defines the API routes and middleware. The backend validates incoming requests, verifies JWTs for protected routes, applies task-related logic, and interacts with MongoDB. Authentication and authorization checks must be performed on the server, including ownership checks for user-specific records.

6.3 Database Integration

MongoDB stores the application data. A logical design may include user and task records, with each task associated with its owner. A task may contain a title, description, due date, priority, status, and timestamps, depending on the implemented schema. The final report should use the actual model fields from the repository.

6.4 API Integration

HTTP Method	Illustrative route	Purpose
POST	/api/auth/register	Register an account, if supported
POST	/api/auth/login	Authenticate and issue token, if supported
GET	/api/tasks	Retrieve user task records
POST	/api/tasks	Create a task
GET	/api/tasks/:id	Retrieve a task
PUT/PATCH	/api/tasks/:id	Update a task
DELETE	/api/tasks/:id	Delete a task

These routes are examples for documentation and must be replaced with the actual Express route paths before submission.

6.5 Scheduler and Notification Integration

Node-Cron is intended to execute periodic deadline checks. The scheduler evaluates task due dates and reminder conditions. Nodemailer is used to send email notifications through configured mail transport settings. Socket.IO provides event-based updates to connected clients. The implementation should record or otherwise control reminder state to avoid duplicate messages; this should be claimed only if supported by the code.

6.6 Development Tools

GitHub supports version control and team collaboration. Postman is used to send API requests and inspect response status codes and payloads. Docker can package application services into repeatable environments when Dockerfile or Compose configuration is available.

7. RESULTS AND DISCUSSION

This section is intended to present evidence from the completed application. Since actual runtime screenshots and test logs have not been supplied with the reference document, the following are figure placeholders and evaluation descriptions. Replace each placeholder with a screenshot captured from the running Team 11 application. Do not submit conceptual diagrams as proof of implemented functionality.

Fig. 1. Application landing page / dashboard

Insert a screenshot of the actual React dashboard. The screenshot should show the navigation and task summary visible after a successful login.

Fig. 2. User authentication screen

Insert the actual login or registration screen. Briefly explain the fields and the authentication response.

Fig. 3. Task creation form

Insert the task form with representative task information. Explain how a user enters task details and a due date.

Fig. 4. Task list and status view

Insert the actual task list. Explain how task records, priorities, due dates, and status are presented.

Fig. 5. Deadline notification

Insert evidence of the actual email reminder or in-app Socket.IO event. Explain the trigger condition and notification channel.

Fig. 6. API testing using Postman

Insert a Postman screenshot showing a real endpoint, request method, response status, and response body.

Fig. 7. Database records / deployment

Insert a screenshot of relevant MongoDB records or Docker services, if used. Explain what the evidence demonstrates.

7.1 Test Case Summary

Test case	Expected result	Actual result
Valid authentication	User is authenticated	To be filled after testing
Protected route without token	Request is rejected	To be filled after testing
Create task	Task is stored and returned	To be filled after testing
Update/delete task	Authorized operation succeeds	To be filled after testing
Deadline reminder	Eligible task triggers configured reminder	To be filled after testing
Email notification	Mail transport attempts delivery	To be filled after testing
Real-time event	Connected client receives relevant event	To be filled after testing

Discussion: Report the observed behavior, test environment, and any failures. Add numerical values such as response time or notification delay only when they have been measured repeatedly and the measurement method is stated.

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The Service-Oriented Task & Deadline Management Platform is designed to provide a centralized web-based environment for organizing tasks and monitoring due dates. The proposed technology stack combines React.js and Tailwind CSS for the user interface, Node.js and Express.js for backend services, MongoDB for data persistence, and JWT for authenticated access. Node-Cron supports scheduled deadline evaluation, while Nodemailer and Socket.IO provide email and real-time notification capabilities.

The project emphasizes modular responsibilities and REST-based communication between the frontend and backend. The separation of task operations, authentication, persistence, scheduling, and notification handling provides a maintainable foundation for further development. Final claims about successful implementation and system performance should be supported by actual application screenshots and test results.

8.2 Future Scope

Near-Term Enhancements

- Add configurable reminder intervals and user notification preferences.
- Improve validation messages and task filtering options.
- Maintain notification history and delivery status, if required.
- Add automated tests for authentication, task operations, and reminder behavior.

Medium-Term Enhancements

- Introduce recurring tasks and calendar synchronization.
- Support shared projects and task assignment with role-based access control.
- Provide analytics for completed, pending, and overdue tasks.
- Add robust retry handling for temporary email delivery failures.

Long-Term Enhancements

- Develop a Progressive Web App or mobile client.
- Introduce scalable background-job processing if task volume increases.
- Add integrations with external productivity platforms.
- Evaluate the application under larger concurrent workloads and document measured performance.

9. REFERENCES

[1] Todoist, “Todoist – Organize your work and life,” https://todoist.com/ (accessed Oct. 2026).

[2] Microsoft, “Microsoft To Do,” https://to-do.office.com/ (accessed Oct. 2026).

[3] Trello, “Trello Project Management,” https://trello.com/ (accessed Oct. 2026).

[4] Asana, “Work Management Platform,” https://asana.com/ (accessed Oct. 2026).

[5] ClickUp, “Project Management and Productivity,” https://clickup.com/ (accessed Oct. 2026).

[6] Any.do, “Task Management and Planner,” https://www.any.do/ (accessed Oct. 2026).

[7] Google, “Google Tasks,” https://tasks.google.com/ (accessed Oct. 2026).

[8] Notion Labs, “Notion Workspace,” https://www.notion.so/ (accessed Oct. 2026).

[9] Atlassian, “Jira Software,” https://www.atlassian.com/software/jira (accessed Oct. 2026).

[10] TickTick, “TickTick Task Management,” https://ticktick.com/ (accessed Oct. 2026).

[11] React, “React Documentation,” https://react.dev/learn (accessed Oct. 2026).

[12] OpenJS Foundation, “Node.js Documentation,” https://nodejs.org/docs/ (accessed Oct. 2026).

[13] Express.js, “Express Documentation,” https://expressjs.com/ (accessed Oct. 2026).

[14] MongoDB, “MongoDB Manual,” https://www.mongodb.com/docs/manual/ (accessed Oct. 2026).

[15] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015.

[16] Socket.IO, “Socket.IO Documentation,” https://socket.io/docs/v4/ (accessed Oct. 2026).

[17] Nodemailer, “Nodemailer Documentation,” https://nodemailer.com/ (accessed Oct. 2026).

[18] Docker, “Docker Documentation,” https://docs.docker.com/ (accessed Oct. 2026).

[19] Postman, “Postman Learning Center,” https://learning.postman.com/ (accessed Oct. 2026).

[20] OWASP Foundation, “OWASP API Security Top 10,” https://owasp.org/www-project-api-security/ (accessed Oct. 2026).

APPENDIX A – DETAILED SYSTEM ARCHITECTURE

This appendix expands the architecture description in Section 6. It explains the responsibilities and interactions of the main components using the technology stack specified by Team 11. The descriptions are architectural documentation; exact module names, route names, deployment topology, and implementation choices must be confirmed from the project repository.

A.1 Architectural Layers

Layer	Technology / component	Responsibilities
Presentation	React.js, Tailwind CSS	Renders screens, accepts user input, presents task and notification information.
Communication	REST API over HTTP	Transfers requests and responses between the browser and backend.
Application	Node.js, Express.js	Routes requests, applies validation and task-related rules, coordinates services.
Security	JWT middleware	Verifies authentication tokens for protected operations.
Persistence	MongoDB	Stores user and task data according to the implemented schemas.
Background processing	Node-Cron	Runs scheduled deadline-evaluation work.
Notification	Nodemailer, Socket.IO	Provides email delivery and real-time events.
Packaging / delivery	Docker, GitHub	Supports container packaging and source-code collaboration.

A.2 Request-Response Interaction

A typical user operation begins in the React interface. The client gathers the required fields and sends a request to an Express endpoint. The backend applies request validation and, for protected routes, verifies the JWT. After authentication, the application checks whether the requested record belongs to the current user. The task operation then reads or modifies the relevant MongoDB document and returns a response to the browser. The frontend uses that response to refresh the displayed information.

A.3 Background Notification Interaction

Deadline processing is separate from an ordinary browser request. Node-Cron initiates a scheduled execution according to the configured schedule. The scheduler obtains tasks that may qualify for a reminder and evaluates the deadline condition. If a task meets the configured rule, the notification component prepares the message and invokes Nodemailer. If the user is connected, Socket.IO may also emit an event to the relevant client. The exact reminder window, event name, and duplicate-prevention mechanism depend on the implementation.

A.4 Architectural Qualities

- Modularity: each major responsibility has a defined role, reducing unnecessary coupling.
- Maintainability: frontend presentation can be updated independently of persistence logic when API contracts remain stable.
- Security: authentication and resource-ownership checks are performed by the backend rather than trusted to the browser.
- Recoverability: a client that misses a real-time event should retrieve current state through the REST API after reconnecting.
- Extensibility: additional notification channels or task attributes can be introduced through explicit interfaces.

APPENDIX B – DATA DESIGN AND VALIDATION

The project description identifies MongoDB as the persistence technology. The examples in this appendix describe a possible logical data design and are not a substitute for the actual Mongoose models or database definitions. Before submission, replace the illustrative field names with the exact schema used by the application.

B.1 Logical Entities

Entity	Illustrative attributes	Purpose
User	_id, name, email, passwordHash, createdAt	Represents an account that can access the application.
Task	_id, ownerId, title, description, dueDate, priority, status, createdAt, updatedAt	Represents a unit of work and its deadline-related details.
Notification (optional)	_id, userId, taskId, type, sentAt, deliveryStatus	Could record reminder attempts if notification history is implemented.

B.2 Data Validation Rules

Field / data	Validation consideration
Title	Required if the task form requires a title; reject blank or whitespace-only values.
Description	Apply a maximum length appropriate to the interface and database model.
Due date	Accept a valid date representation and apply a consistent timezone policy.
Priority	Restrict to the allowed values defined by the application.
Status	Restrict to supported lifecycle values.
Owner identifier	Derive from the verified authenticated identity, not an untrusted request field.
Email	Validate format and apply uniqueness only if required by the account model.

B.3 Data Integrity and Ownership

User-specific task queries should include the authenticated owner's identifier. When a user requests a task by its database identifier, the backend should verify both that the task exists and that the requester is authorized to access it. An unguessable identifier alone is not an authorization mechanism. Updates should validate the incoming fields and should not allow a request to overwrite ownership information unless a controlled administrative operation explicitly permits it.

B.4 Date and Time Handling

Deadline systems require consistent date interpretation. The application should define whether due dates are stored in UTC and converted to the user's local timezone for display, or whether another consistent convention is used. The scheduler and frontend must apply the same interpretation. Boundary cases include a task due exactly at a reminder threshold, a task whose deadline has passed, and a task completed before the scheduler executes.

B.5 Indexing Considerations

If task lists are frequently retrieved by owner and sorted by due date, a compound index may be considered after reviewing the actual query patterns. Indexes should be chosen based on measured use rather than added indiscriminately. For the academic prototype, correctness of ownership filtering and data validation takes priority over speculative performance optimization.

APPENDIX C – REST API DESIGN REFERENCE

The API paths below are illustrative examples based on common task-management operations. They are not verified routes from the Team 11 source code. Replace them with the actual Express router definitions and include real Postman request and response captures in the final submission.

C.1 Example Endpoint Catalogue

Method	Example path	Purpose	Auth
POST	/api/auth/register	Create an account, if supported	No
POST	/api/auth/login	Authenticate and return token, if supported	No
GET	/api/tasks	List current user's tasks	Yes
POST	/api/tasks	Create a task	Yes
GET	/api/tasks/:id	Retrieve a task	Yes
PATCH	/api/tasks/:id	Update selected task fields	Yes
DELETE	/api/tasks/:id	Delete a task	Yes

C.2 Request Headers

A protected request commonly includes an Authorization header using the Bearer token scheme. The token should be sent over HTTPS in a deployed environment. The backend must verify the signature and relevant claims before allowing access.

```
Authorization: Bearer <JWT>
```

C.3 Illustrative Create-Task Payload

```
{
  "title": "Complete SOA project report",
  "description": "Review documentation and add implementation evidence",
  "dueDate": "2026-11-05T18:00:00.000Z",
  "priority": "high",
  "status": "pending"
}
```

C.4 Response and Error Handling

HTTP status	Typical interpretation	Client behavior
200	Operation completed	Display returned resource or confirmation.
201	Resource created	Refresh list or navigate to created resource.
400	Invalid request data	Show field-level or general validation message.
401	Missing or invalid authentication	Request sign-in or refresh authentication.
403	Authenticated but not permitted	Show an access-denied message.
404	Resource not found / not visible	Show a not-found state.
500	Unexpected server failure	Show a safe generic message and retain diagnostics in server logs.

C.5 Postman Validation Procedure

1. Create a Postman collection for the implemented API routes.
2. Set the backend base URL in a collection or environment variable.
3. Test public authentication endpoints using valid and invalid inputs.
4. Obtain a JWT and configure it as a Bearer token for protected requests.
5. Test successful task operations and invalid payloads.
6. Test access control by attempting to access a task belonging to a different account.
7. Save representative request and response examples for the project report.

APPENDIX D – SCHEDULING AND NOTIFICATION WORKFLOW

D.1 Scheduler Responsibilities

Node-Cron is included in the project stack for scheduled execution. The scheduler should run at a defined interval and invoke a deadline-checking function. That function identifies tasks that meet the configured reminder rule and excludes tasks that should not be notified, such as completed tasks, if that is the intended application policy.

D.2 Reminder Evaluation Sequence

8. The scheduled job starts according to the configured cron expression.
9. The backend queries tasks that may fall within the reminder window.
10. The reminder rule checks due date, task status, and any stored reminder state.
11. An eligible task is passed to the notification workflow.
12. Nodemailer attempts email delivery using the configured mail transport.
13. Socket.IO emits a user-scoped event when the relevant client is connected.
14. The system records the outcome if notification tracking is implemented.

D.3 Duplicate Notification Risk

A scheduler runs repeatedly, so a task may remain eligible across more than one execution. Without a duplicate-prevention strategy, the same message could be sent multiple times. A design may store a sent timestamp, a reminder flag, or a notification record. If multiple scheduler instances can run concurrently, the eligibility check and state update should be designed to reduce race conditions. The report should describe only the mechanism actually present in the code.

D.4 Email Delivery Considerations

- Use a test mailbox during development and demonstrations.
- Keep SMTP credentials in environment configuration, not source files.
- Handle transport errors so that a mail failure does not corrupt the task record.
- Use clear subject lines and include task title and due-date information where appropriate.
- Do not claim guaranteed delivery; the external mail provider controls final delivery.

D.5 Socket.IO Event Considerations

Socket.IO supports low-latency event delivery to connected clients. The server should associate connections with verified users and emit private events only to the correct user or room. A browser may disconnect or miss events. Therefore, the REST API remains the authoritative way to retrieve current task state after reconnecting.

D.6 Timezone and Boundary Cases

Scenario	Expected policy to define
Task due within reminder window	Determine whether one reminder or repeated reminders are intended.
Task due exactly at threshold	Define inclusive or exclusive comparison.
Task already completed	Normally exclude from pending deadline reminders, if completion is final.
Task overdue	Define whether overdue reminders are sent and how often.
Mail transport unavailable	Handle and record failure; define whether retry is supported.
Client offline	Do not assume real-time event delivery; refresh state on reconnect.

APPENDIX E – TEST PLAN AND EVALUATION

This test plan provides a structured way to evaluate the application. It is not a statement that the tests have already passed. Fill the actual-result and status columns after running the application and retain screenshots or logs as evidence.

ID	Test scenario	Expected result	Actual / status
T01	Load frontend application	Main page loads without a fatal client error	To be recorded
T02	Submit valid login details	Authenticated session/token is created	To be recorded
T03	Submit invalid credentials	Authentication is rejected safely	To be recorded
T04	Call protected API without token	Request is rejected	To be recorded
T05	Create task with valid fields	Task is stored and returned	To be recorded
T06	Create task with missing required field	Validation error is returned	To be recorded
T07	Update a task owned by current user	Authorized update succeeds	To be recorded
T08	Access another user's task	Access is denied	To be recorded
T09	Run scheduler with eligible task	Task is evaluated by reminder rule	To be recorded
T10	Trigger email reminder	Mail transport attempts delivery	To be recorded
T11	Connect Socket.IO client	Client receives intended event	To be recorded
T12	Disconnect and reconnect client	Client can retrieve current state	To be recorded

E.1 Evaluation Metrics

Metric	How it can be measured
API response time	Measure request start to response receipt across repeated requests.
Task-operation success rate	Successful valid operations divided by total valid test attempts.
Scheduler detection delay	Difference between the expected reminder time and scheduler detection time.
Email attempt outcome	Count successful transport submissions and failed attempts.
Real-time event delay	Measure event emission to client receipt while connected.

E.2 Evidence Checklist

- Screenshot of the running dashboard and task list.
- Screenshot of task creation and update behavior.
- Postman request and response for authentication and task endpoints.
- Server log showing a scheduler execution.
- Test email or mail-provider evidence for reminder delivery.
- Browser evidence showing a Socket.IO event.
- Test case table completed with actual observations.

E.3 Reporting Results Responsibly

Record the machine, runtime version, database environment, network conditions, test data size, and number of repetitions when reporting measurements. Avoid generalizing results from a small local demonstration to production-scale behavior. If a test is not implemented or cannot be executed, mark it as not tested rather than reporting a pass.

APPENDIX F – INSTALLATION, CONFIGURATION AND DEPLOYMENT

F.1 Development Prerequisites

- A compatible Node.js runtime and npm installation.
- Git for cloning and managing the repository.
- MongoDB local instance or an accessible MongoDB deployment.
- Email-provider credentials if the reminder feature is enabled.
- Postman for API testing.
- Docker Desktop or Docker Engine if container configuration is supplied.

F.2 Repository Setup

```
git clone <repository-url>
cd <repository-folder>
# Install dependencies in each folder containing package.json
npm install
```

The actual repository may contain separate frontend and backend directories. Run dependency installation in the relevant directories and use the scripts defined in each package.json. Do not assume that a particular folder name or npm script exists until it has been checked.

F.3 Environment Configuration

```
PORT=<backend-port>
MONGODB_URI=<database-connection-string>
JWT_SECRET=<long-random-secret>
EMAIL_HOST=<smtp-host>
EMAIL_PORT=<smtp-port>
EMAIL_USER=<mail-account>
EMAIL_PASS=<mail-app-password>
CLIENT_URL=<frontend-origin>
```

These names are illustrative. The actual application may use different environment-variable names. Never commit real credentials, JWT secrets, or private connection strings to GitHub.

F.4 Running the Application

```
# Use the scripts defined in package.json
npm run dev
# or, where configured
npm start
```

F.5 Docker Deployment

If Dockerfiles or a Compose file are present, document each service, build context, port mapping, volume, dependency, and environment variable from those files. Containerization can make runtime setup more repeatable, but a running container does not guarantee that external services such as MongoDB or SMTP are reachable. Verify application health after startup.

F.6 Deployment Checklist

- Verify the frontend API base URL points to the intended backend.
- Verify the backend can connect to MongoDB.
- Verify JWT secrets and mail credentials are configured securely.
- Ensure the scheduler is started only as intended; multiple instances can cause duplicate scheduled work.
- Test email delivery using a controlled recipient.
- Verify Socket.IO CORS and authentication settings.
- Review logs for startup errors without exposing secrets.

F.7 Troubleshooting Reference

Symptom	Checks
Frontend cannot reach API	Backend process, API URL, CORS, network panel.
Database connection error	MongoDB URI, credentials, network allowlist, service availability.
JWT rejected	Token expiry, signing secret, Authorization header format.
Email not received	SMTP host/port, credentials, provider restrictions, server logs.
No socket event	Connection state, event name, authenticated room mapping.
Repeated reminders	Scheduler frequency, reminder state, duplicate prevention.

F.8 Step-by-Step Troubleshooting

When a problem occurs during development or demonstration, diagnose it layer by layer instead of changing several components at once. First confirm that the frontend development server is running and that the browser can load the application. Next inspect the browser network panel to identify the exact request that failed, including its URL, HTTP method, status code, and response body. Then inspect backend logs for the corresponding request and determine whether the failure occurred during routing, authentication, validation, database access, or notification processing.

For a database connection failure, verify that the database service is running and that the configured connection string is valid. Check whether the database host is reachable from the backend environment and whether credentials have the required permissions. If the application uses a hosted database, verify its network access rules. Avoid printing the full connection string in screenshots or logs because it may contain credentials.

For authentication failures, inspect the token creation and verification paths. Confirm that the client sends the token in the expected Authorization header format and that the backend uses the same signing configuration that was used to issue the token. Check token expiry and ensure that the client does not continue using an old token after sign-out or account changes. Never paste a live JWT into the project report.

For email problems, first test the configured mail transport independently from the scheduled workflow if the project supports such a test. Confirm host, port, secure-connection settings, account credentials, and provider-specific restrictions. A successful request to the backend does not necessarily prove that an email reached the recipient's inbox. Record whether the application handed the message to the transport and distinguish that from confirmed delivery.

For real-time notification problems, verify that the browser establishes a Socket.IO connection and that the server accepts the connection under the configured origin and authentication policy. Confirm that the server and client use matching event names and payload shapes. If the event is scoped to a user room, verify that the authenticated user is joined to the correct room. After reconnecting, retrieve the current task state through the REST API.

For duplicate reminders, inspect the scheduler frequency, the task-selection query, and the reminder-state update. Check whether multiple backend instances are running the same scheduled job. If the design stores a sent timestamp or notification state, confirm that it is updated at the correct point in the workflow and that failures are handled consistently.

F.9 Demonstration Readiness Checklist

- Start the frontend and backend using the documented commands and confirm that both remain running.
- Verify database connectivity before demonstrating task operations.
- Use a test account and test data rather than exposing personal information.
- Confirm that create, view, update, and delete operations behave as expected in the actual application.
- Use a controlled due date or test interval to demonstrate scheduler behavior safely.
- Use a test email recipient and verify the configured notification behavior.
- Keep Postman collections and environment variables free of passwords and live tokens in screenshots.
- Check that the presentation laptop has the required network access and environment configuration.
- Prepare a fallback screenshot or recorded evidence in case an external mail provider is temporarily unavailable.

F.10 Maintenance Recommendations

After each project change, run a short regression check covering authentication, task operations, and the reminder workflow. Keep the README synchronized with the actual installation steps and environment-variable names. Review dependency updates before adopting them, and retain a known-working version for the project demonstration. Back up any important demonstration data and remove test accounts or credentials that should not remain in the final deployment.

The final handover should include the source repository, setup instructions, environment-variable template without secrets, API collection, database model descriptions, and evidence screenshots. This material allows another student or evaluator to reproduce the application behavior without relying on undocumented steps.